

PROJECT DESIGN PHASE

AI BlogNest API

Project Name:	AI BlogNest API
Team ID:	SWTID-2026-6999
Date:	22 September 2026

1. Design Overview

AI BlogNest API is designed as a modular backend REST API for intelligent blog content management. The design separates API routing, request validation, authentication, authorization, business logic, database operations, AI processing, and search processing into independent components.

The system uses Node.js and Express.js as the backend platform, MongoDB Atlas as the database, Google Gemini for AI processing, MongoDB Atlas Search for keyword-based search, and MongoDB Atlas Vector Search for semantic search.

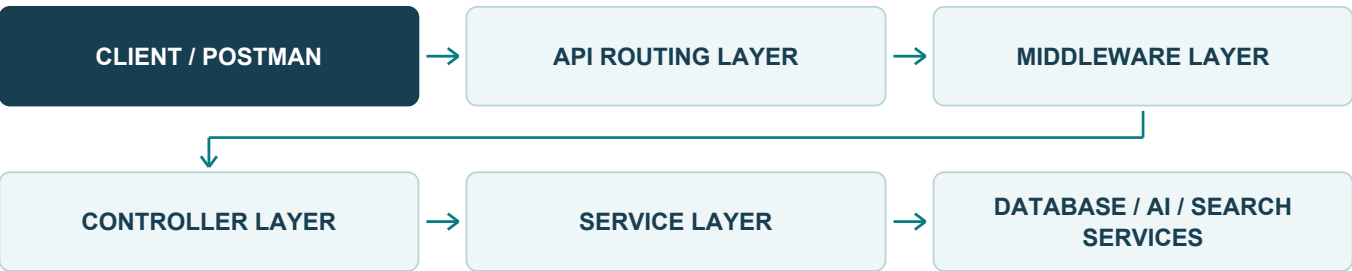
The design follows the MVC pattern with an additional service layer to keep business logic separate from controllers. The BlogNest specification describes this layered architecture and identifies the AI generation and summarization services as part of the backend design.

2. Design Objectives

- The system is designed to:
- Provide secure REST API access
 - Separate functionality according to user roles
 - Manage the complete blog lifecycle
 - Maintain relationships between users, blogs, categories, comments, and embeddings
 - Integrate Gemini for AI content generation and summarization
 - Provide keyword-based and semantic search
 - Protect application resources through authentication and authorization
 - Support maintainable and scalable backend development

3. High-Level System Design

The overall system is divided into the following layers:



Main Components

API Layer	Security Layer	Business Layer	Data Layer
Authentication APIs User APIs Blog APIs Category APIs Comment APIs AI APIs Search APIs	JWT authentication Role authorization Validation Sanitization Rate limiting	Blog processing Category processing Comment processing AI processing Search processing	Users Blogs Categories Comments Embeddings

4. Module Design

4.1 Authentication Module

Responsible for:

- User registration
- User login
- JWT generation
- Token verification
- Authenticated-user identification

4.2 User and Role Management Module

Responsible for:

- Maintaining user information
- Maintaining user roles
- Assigning roles through authorized administration
- Enforcing role-based permissions

Supported roles: Admin | Editor | Author | Reader

4.3 Blog Management Module

Responsible for: creating, retrieving, updating and deleting blogs; managing blog status; category assignment; tag management; and author ownership.

Blog Lifecycle



The BlogNest specification defines blog lifecycle states and ownership-based management.

4.4 Category Module

Responsible for creating, viewing, updating and deleting categories, and organizing blog content.

4.5 Comment Module

Responsible for creating, retrieving, updating, deleting and moderating comments.

Comment States



The BlogNest design specifically includes comment moderation as part of the content-management workflow.

4.6 AI Module

Responsible for generating blog content, generating structured blog information, summarizing blog content, and processing AI prompts.

Main operations: Generate Blog | Summarize Blog

The project specification defines the AI endpoints `/api/ai/generate-blog` and `/api/ai/summarize`.

4.7 Search Module

The search module contains two different search mechanisms:

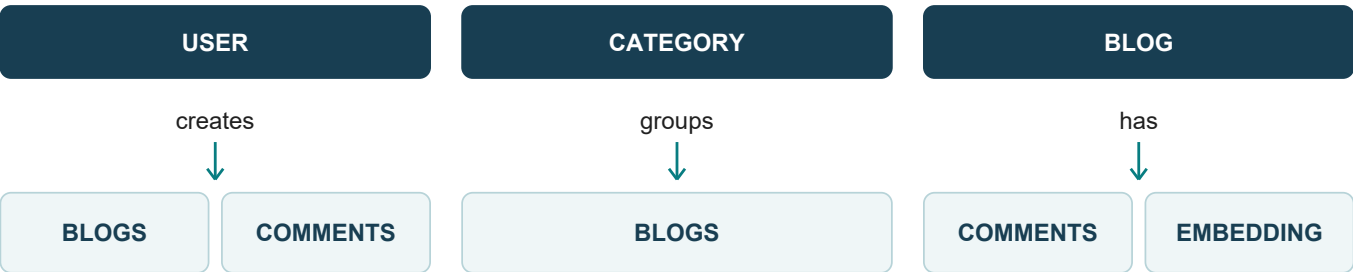
- Keyword Search - uses MongoDB Atlas Search to find blogs using words or phrases.
- Semantic Search - uses embeddings and MongoDB Atlas Vector Search to retrieve content based on meaning.

These capabilities are also part of the common TN Skills requirements.

5. Database Design

The system uses the following collections: Users, Blogs, Categories, Comments, Embeddings.

Relationships



A user can create multiple blogs and comments. A blog can contain multiple comments and is associated with category and embedding information. The BlogNest specification describes the primary User, Post, and Comment relationships.

6. API Design

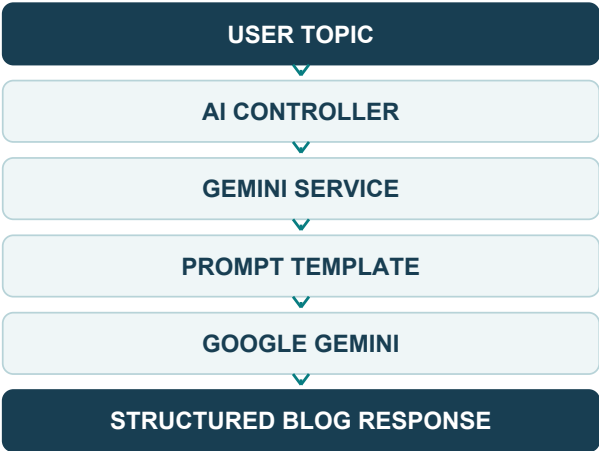
The backend is organized into resource-based REST endpoints.

Authentication	
POST	/api/auth/register
POST	/api/auth/login
POST	/api/auth/logout
GET	/api/auth/me
Users	
GET	/api/users
GET	/api/users/profile
PUT	/api/users/:id
PATCH	/api/users/:id/role
Blogs	
POST	/api/blogs
GET	/api/blogs
GET	/api/blogs/:id
PUT	/api/blogs/:id
DELETE	/api/blogs/:id
Categories	
POST	/api/categories
GET	/api/categories
GET	/api/categories/:id
PUT	/api/categories/:id
DELETE	/api/categories/:id
Comments	
POST	/api/blogs/:blogId/comments
GET	/api/blogs/:blogId/comments
PUT	/api/comments/:id
DELETE	/api/comments/:id
AI	
POST	/api/ai/generate-blog
POST	/api/ai/summarize
Search	
GET	/api/search?q=
GET	/api/search/semantic?q=

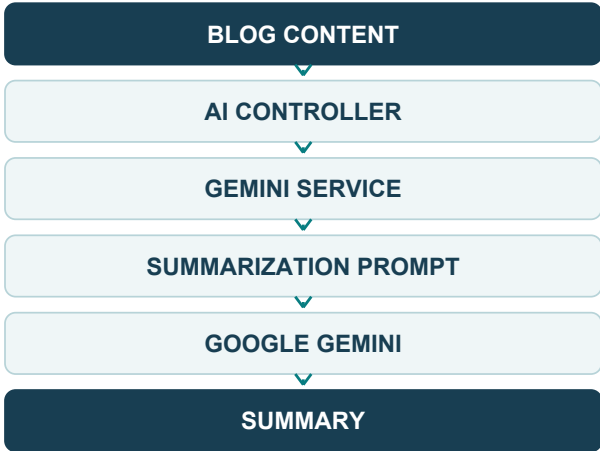
7. AI Design

The AI subsystem is designed as a separate service so that AI processing does not become part of the route or controller logic.

AI Blog Generation



AI Summarization



8. Prompt Design

The system uses structured prompts to control the AI response.

For blog generation, the prompt is designed to request:

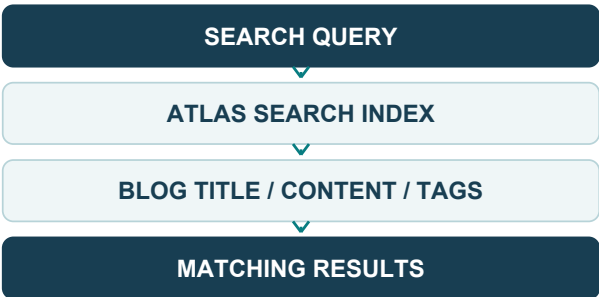
- Appropriate title
- Structured content
- Summary
- Relevant tags

For summarization, the prompt instructs the model to produce a concise representation of the provided blog content.

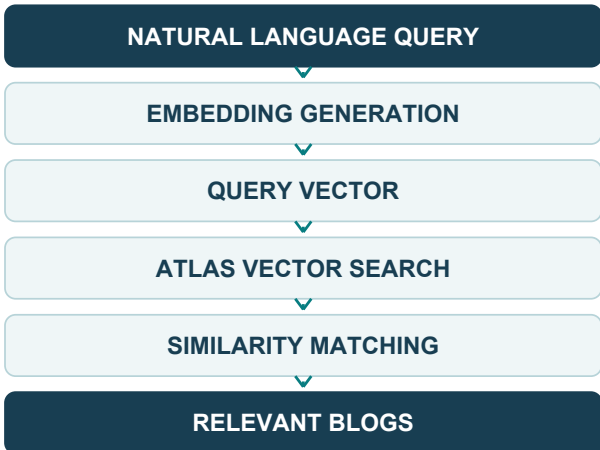
This satisfies the Prompt Engineering requirement in the common TN Skills checklist.

9. Search Design

9.1 Keyword Search



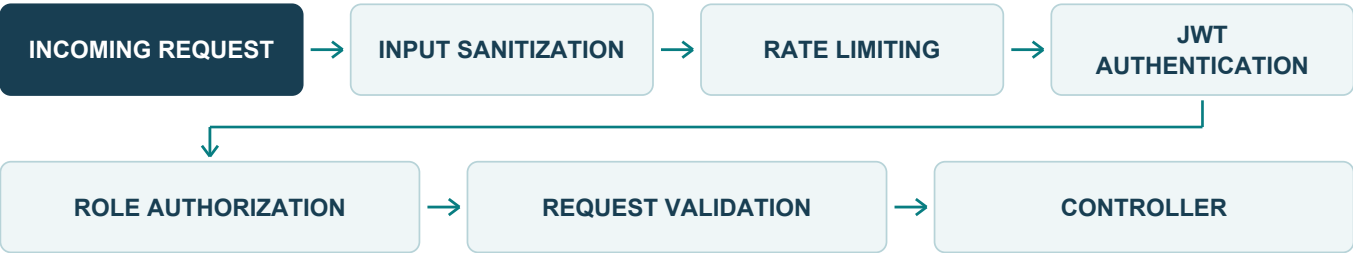
9.2 Semantic Search



The semantic-search design uses embeddings to represent both stored blog content and user queries and retrieves semantically related content.

10. Security Design

Security is incorporated into the request-processing flow.

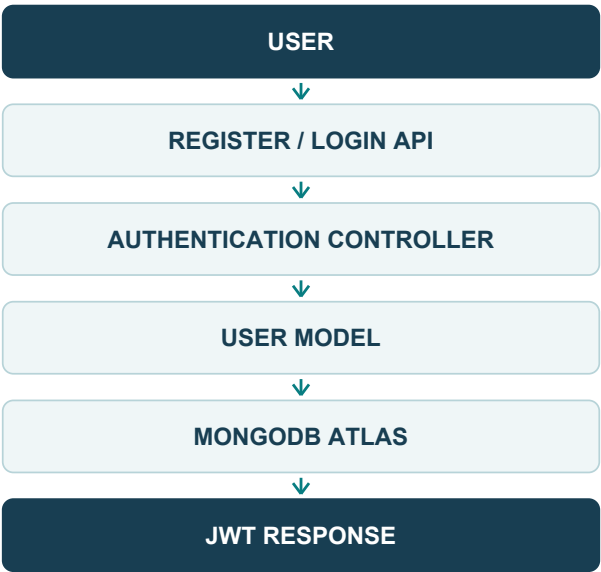


Security mechanisms include:

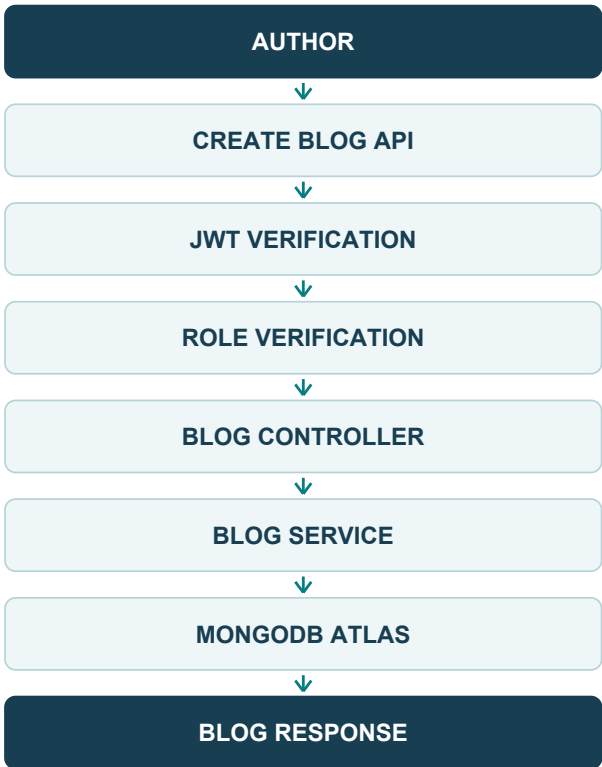
- bcrypt password hashing
- JWT authentication
- Role-based authorization
- Input validation
- Input sanitization
- Rate limiting
- Safe error handling

11. Data Flow Design

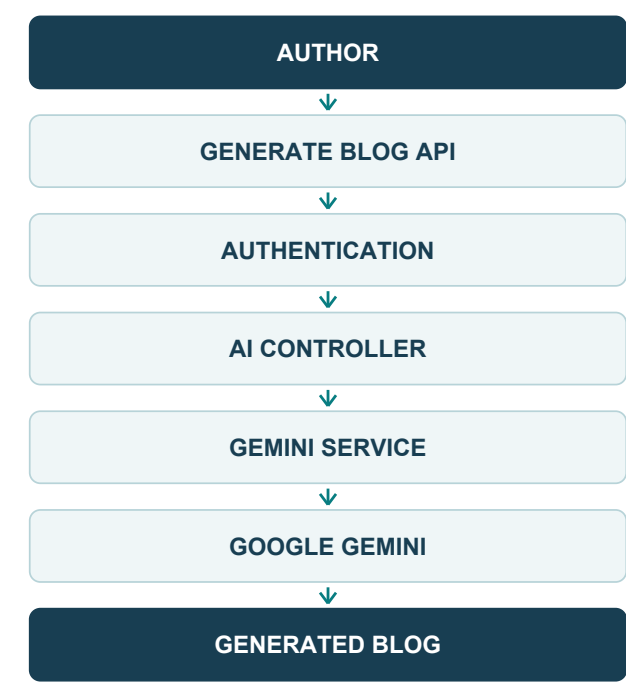
User Authentication Flow



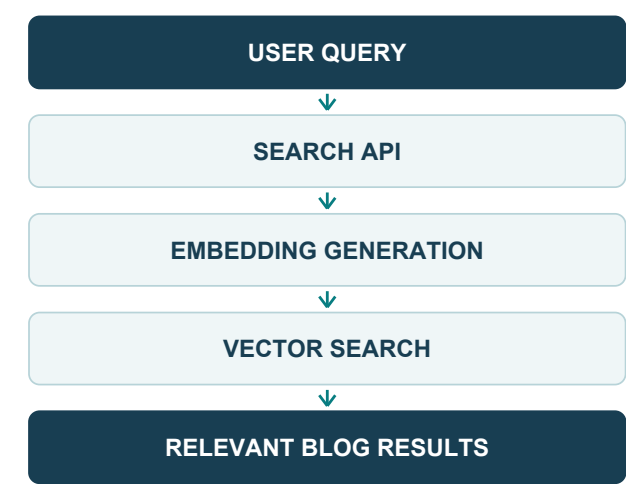
Blog Creation Flow



AI Generation Flow



Semantic Search Flow



12. Role-Based Design

The authorization design uses the four project roles.

Role	Main Access
Admin	Complete system management
Editor	Content review, publishing and moderation
Author	Blog creation and own-content management
Reader	Published content and comments

The BlogNest project defines these four roles and their corresponding responsibilities.

13. Error Handling Design

The system uses centralized error handling. The API returns consistent responses.

Successful Response

```
{
  "success": true,
  "message": "Operation successful",
  "data": {}
}
```

Error Response

```
{
  "success": false,
  "message": "Operation failed"
}
```

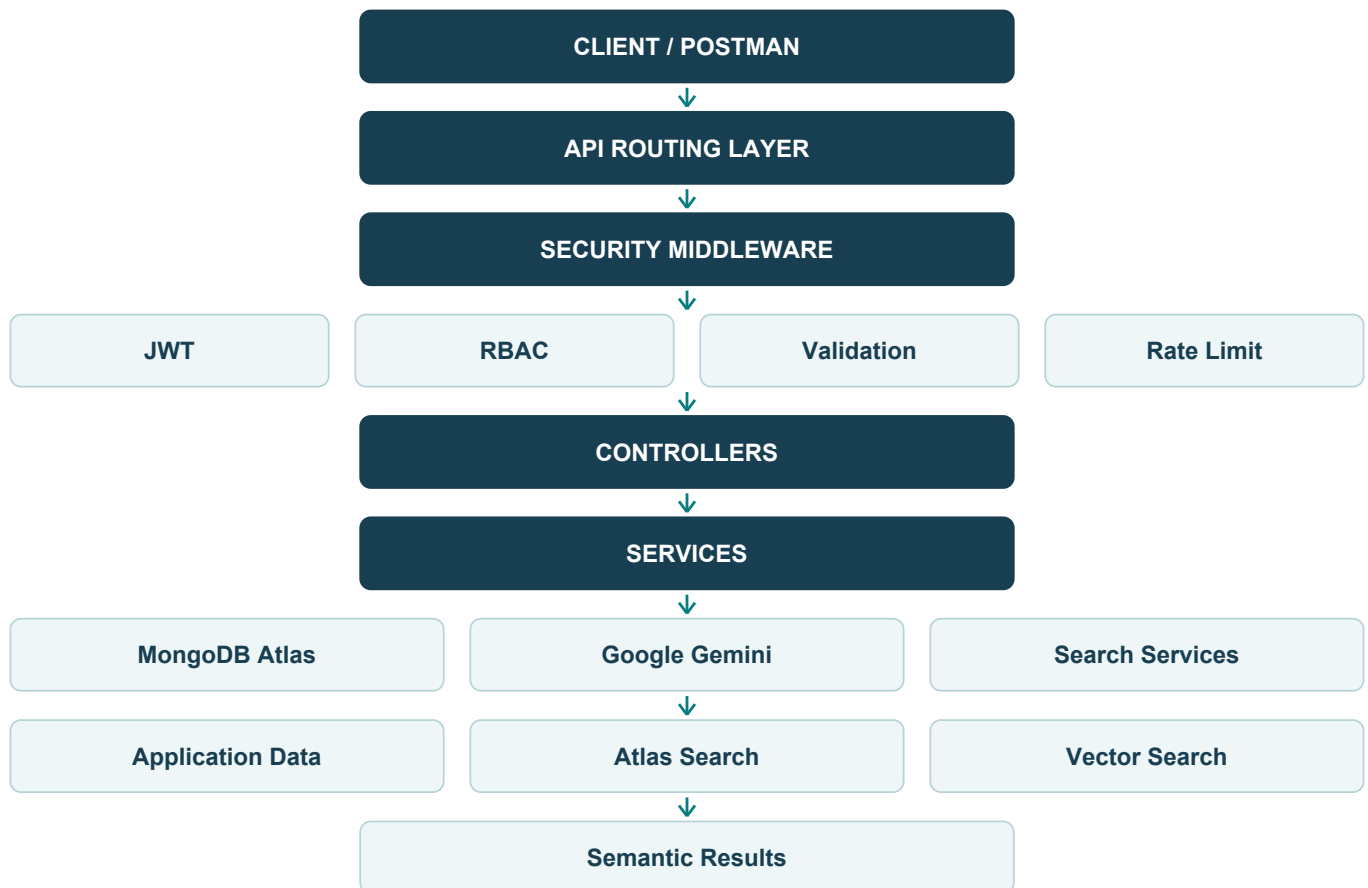
The design ensures that validation errors, authentication failures, authorization failures, missing resources, and server errors are handled consistently.

14. Scalability Design

The system uses modular components so that each major function can be extended independently.

- AI functionality can be extended with additional Gemini services
- Search functionality can support additional searchable fields
- Additional user roles can be introduced through RBAC
- Additional content-management modules can be added through separate controllers and services
- MongoDB Atlas provides cloud-based database infrastructure

15. Complete Design Flow



16. Design Outcome

The final design provides a clear separation between:

- Client interaction
- API routing
- Security
- Business logic
- Database operations
- AI processing
- Search processing

This design allows AI BlogNest to function as a secure, modular backend REST API while satisfying the project-specific BlogNest requirements and the common TN Skills technical requirements.